

Attack Your Own Agent’s Authorization: Testing MCP and OAuth Agent Tokens Before Deployment

Anonymous Submission

Short position / work-in-progress paper (double-blind review)

Abstract—An autonomous agent acts on a bearer token. If an attacker can steer that agent through untrusted input, such as a poisoned document or a malicious tool description, the attacker gains whatever the token permits. Authorization is therefore part of an agent’s attack surface, and it has one useful property that the model’s behavior does not: a defender can test it adversarially before deployment. This paper takes that offensive view and uses it to sort the defensive problem. We place fifteen documented agent-token threats into two groups by a single question: can a defender reproduce the attack offline, from one token or server manifest? Six can be reproduced this way. The other nine cannot, because they depend on a live runtime, such as a consent flow, a human approver, or a workload-identity layer that most deployments do not yet run. For the six, we release an open-source generator that mints adversarial agent tokens and an analyzer that rejects them, with every finding cited to the OAuth RFCs, the MCP authorization specification, or the OWASP Agent Top 10. Both run in CI or next to the agent, so a team can red-team its own authorization before shipping. The nine that cannot be tested offline set out a runtime-defense agenda. This is work in progress: the instrument is implemented, and a prevalence measurement across public deployments is ongoing and is not reported here.

1. Introduction

A tool-using agent presents a bearer token on every call it makes for a user, either an OAuth access token or an MCP authorization token. The agent is autonomous, and its inputs are not. It reads web pages, summarizes documents, and follows tool descriptions, any of which an attacker may control. Prompt injection does not need a memory-safety bug to cause damage. It needs a token that grants more than the current action requires.

The resulting failures are ordinary OAuth failures, and the agent’s autonomy makes them easier to reach [1], [2]. A token that is not bound to the server it is sent to can be replayed against a different server, or passed through it. An on-behalf-of token with no verifiable delegation chain leaves the question “who authorized this?” unanswerable. A wildcard scope such as `admin:*` gives an injected agent far more access than any one task needs. A token with no expiry stays useful to whoever obtains it.

Much of this is testable before an agent ever meets an adversary. A defender can mint the same malicious tokens an attacker would forge, then confirm that its own verifier rejects them. That is an inexpensive red-team of the authorization layer, and it can run on every build. It also has a clear limit. Some agent-authorization attacks cannot be decided from a token at all and appear only against a live deployment. Separating the two cases is the work of this paper.

Contributions. First, we sort the agent-authorization threat surface by offensive testability: what a defender can reproduce before deployment, against what only a runtime defense can stop. Second, we release an open-source pair for the testable subset, a generator of adversarial agent tokens and a stateless analyzer that rejects them with cited findings. Third, we argue that the threats a defender cannot test offline are a runtime-defense agenda, not a gap in tooling. This is a position and work-in-progress paper. It reports a method and an instrument, not a measurement.

2. The agent-authorization attack surface

Table 1 lists fifteen threats in six categories. Each is tied to its normative source and to the invariant an attacker breaks. The final column records the distinction this paper is built on: whether a defender can reproduce the attack from a single token or server manifest, which we call pre-deployment testable, or whether the attack only appears against a live, stateful deployment.

Six of the fifteen are pre-deployment testable: MCP-01, MCP-02, DEL-01, DEL-02, SCOPE-01, and SCOPE-02. Each is a property of a token’s own claims, read against a stated policy. The audience it names, the `act` chain it carries, the scopes it requests, and the lifetime it declares are all present in the token, so an attacker forges them offline and a defender reproduces them the same way. The remaining nine are not testable this way. A confused deputy (CD-01, CD-02) is a property of a consent flow and of how a proxy matches `redirect_uri` [2], [10]. Prompt-injection escalation and tool poisoning (AGT-01, AGT-02) are properties of a running agent and the tools it has been given [11], [12]. Identity confusion (AID-01, AID-02) assumes a workload-identity layer [6], [7] that is usually not deployed, so there is nothing to check a token against.

TABLE 1. AGENT-AUTHORIZATION THREATS BY CATEGORY.
“PRE-DEP.”: A DEFENDER CAN MINT THE MALICIOUS INPUT AND CHECK FOR REJECTION OFFLINE, FROM ONE TOKEN OR MANIFEST. OTHERWISE THE ATTACK NEEDS A LIVE RUNTIME (A CONSENT FLOW, A HUMAN APPROVER, OR A WORKLOAD-IDENTITY SUBSTRATE) AND BELONGS TO RUNTIME DEFENSE.

ID	Sev.	Attack / broken invariant	Pre-dep.?
<i>Audience binding</i> (RFC 8707/9728 [3], [4])			
MCP-01	crit	token passthrough / wrong-audience accepted	✓
MCP-02	high	missing resource indicator / unbound audience	✓
<i>Delegation</i> (RFC 8693 [5])			
DEL-01	high	missing or forged act claim	✓
DEL-02	high	delegation-chain / transitive-trust abuse	✓
DEL-03	high	may_act not enforced on exchange	×
DEL-04	med	impersonation where delegation is required	×
<i>Scope</i>			
SCOPE-01	high	over-scoped agent credential	✓
SCOPE-02	high	non-expiring / long-lived token	✓
SCOPE-03	high	agent reuses the human’s root credential	×
<i>Agent identity</i> (WIMSE/SPIFFE [6], [7])			
AID-01	high	session used as authentication	×
AID-02	med	no distinct workload/agent identity	×
<i>Confused deputy</i> (CWE-441 [8], [9])			
CD-01	crit	confused deputy via static client ID	×
CD-02	high	dynamic client registration abuse (RFC 7591 [10])	×
<i>Agency</i> (OWASP [11], [12])			
AGT-01	crit	prompt-injection-driven privilege escalation	×
AGT-02	high	tool poisoning / rug-pull	×

3. Offense: generating adversarial agent tokens

For each testable class we generate the token an attacker would present. Generation is direct because the invariant is explicit. For audience binding, we emit a token whose resource indicator names a different server than the one it is sent to, or names none (MCP-01, MCP-02). For delegation, we emit an on-behalf-of token with a missing, malformed, or over-deep act chain (DEL-01, DEL-02). For scope, we emit a credential that carries a wildcard or an otherwise over-broad scope beyond a declared allowlist (SCOPE-01). For expiry, we emit a token with no exp, or a lifetime past a declared maximum (SCOPE-02).

These are the inputs a real attacker crafts once it can influence what an agent requests, or can replay a token the agent was issued. Generating them on purpose is the offensive half of a pre-deployment red-team: the defender produces the adversary’s tokens and asks whether its own verifier lets any through. A token that is accepted is a confirmed weakness that an attacker could exploit, with no further analysis needed to interpret it. Figure 1 shows one such token and the findings it draws.

```
adversarial agent token (claims)
  iss = https://issuer.example
  sub = agent-7
  scope = "admin:* files:read"
  aud = (absent)    exp = (absent)

analyzer output (each finding cited)
MCP-02  high no audience; token is
        not bound to a resource [RFC 8707]
DEL-01  high act claim missing;
        delegation unverifiable [RFC 8693]
SCOPE-01 high scope "admin:*" is
        over-broad [OWASP Agentic]
SCOPE-02 high no exp; a non-expiring
        credential [RFC 8725]
```

Figure 1. One adversarial agent token and the analyzer’s findings. The generator emits the token; the analyzer rejects it and cites a source for each finding. Run as a CI gate, any token the verifier accepts fails the build.

4. Defense: a deployable analyzer

The defensive half is a stateless analyzer. It decodes a presented agent token and reports findings against a policy, each citing a normative source: audience binding (MCP-01, MCP-02; RFC 8707/9728 and the MCP authorization specification), on-behalf-of delegation (DEL-01, DEL-02; RFC 8693), scope minimization (SCOPE-01; OWASP Agentic), and expiry (SCOPE-02; RFC 8725). It runs on one token, makes no network calls, and so fits either a CI check against fixtures or a client-side check next to the agent. Nothing leaves the operator.

Paired with the generator of Section 3, the analyzer closes a loop a team can run on every build. Mint the adversarial tokens, confirm the verifier rejects each one, and fail the build if any is accepted. We release both components as open source under a permissive license. The repository link is withheld here for anonymous review.

5. Where defense has to live

The split of six against nine is a claim about where a defense has to run, not only about what a tool happens to cover. Classic OAuth machinery, including resource indicators, token exchange, and protected-resource metadata, does model several of the nine runtime threats [3], [5], [4]. Modeling is not verifying, and common MCP deployments rarely enforce these mechanisms, so the threats fall to the deployer, which is where they are usually missed.

The nine resolve into a concrete defense agenda. CD-01 and CD-02 need per-client consent and exact redirect_uri matching at the proxy. DEL-03, DEL-04, and SCOPE-03 need the server to enforce may_act on token exchange and to keep the agent’s credential distinct from the human’s. AID-01 and AID-02 need a deployed workload-identity substrate [6], [7]. AGT-01 and AGT-02 need per-action authorization, or a human in the loop, so that an injection cannot escalate on its own [12]. Every one of these is a runtime property, and none can be settled by reading a token. Knowing which side of the line an attack

sits on tells a team what to automate in CI today and what to treat as an open runtime problem.

6. Status and limitations

This is work in progress. The generator and the analyzer are implemented for the six pre-deployment-testable classes and released, and the loop between them runs in CI and client-side. We do not report a prevalence measurement. A study of how often these weaknesses occur across public MCP servers and reference agent-token deployments is ongoing, with a hand-validated sample, and we will report it, including its skew toward early-adopter deployments, rather than imply it here. Two limitations bound the method. The MCP specification is still changing, so the taxonomy is a dated snapshot that will need revision. The testable line depends on what a token or manifest exposes: a deployment that hides relevant state reduces what any offline red-team can conclude.

7. Related work

Our building blocks are the OAuth 2.0 framework [13] and its security best practice [1], token exchange [5], resource indicators [3], protected-resource metadata [4], dynamic client registration [10], JWT best practices [14], and the MCP authorization specification [2]. The confused-deputy problem is longstanding [9], [8]. Practitioner taxonomies catalog agentic risk [11], [12], and recent drafts address on-behalf-of agent authentication [15] and workload identity [6], [7]. Offensive work on agents has so far concentrated on prompt injection and model manipulation. We instead treat the authorization layer as the thing to attack and defend, and we contribute both the sort by pre-deployment testability and a paired instrument for the testable subset.

Ethics considerations

The work uses only public specifications, advisories, and CVEs. The generator produces adversarial tokens for a defender to fire at its own verifier. It is deny-by-default and documented for staging rather than production use. We probe no third-party live agents or MCP servers and use no real user tokens. The analyzer runs on tokens the operator already holds and sends nothing anywhere. Any future measurement will characterize population prevalence rather than single deployments, and will follow coordinated disclosure for any identifiable high-severity finding. No personal data or human subjects are involved.

Open science

We release the generator, the analyzer, and the machine-readable threat taxonomy, with each threat recorded alongside its invariant and normative sources, under an open license. The offense-and-defense loop is therefore reproducible and the taxonomy is open to inspection. A DOI-pinned artifact will accompany the camera-ready version.

8. Conclusion

Agent authorization is both reachable by attackers and, in large part, testable by defenders. Six of fifteen documented agent-token threats can be exercised adversarially before deployment. We released a generator and an analyzer that let a team run that red-team in CI, and we argued that the nine threats which cannot be tested offline are the runtime-defense agenda for agentic systems.

References

- [1] T. Lodderstedt, J. Bradley, A. Labunets, and D. Fett, “Best current practice for oauth 2.0 security,” RFC 9700, IETF, 2025, <https://datatracker.ietf.org/doc/html/rfc9700>.
- [2] Anthropic and MCP contributors, “Model context protocol: Authorization and security best practices,” MCP Specification 2025-06-18, 2025, https://modelcontextprotocol.io/specification/2025-06-18/basic/security_best_practices.
- [3] B. Campbell, J. Bradley, and M. Jones, “Resource indicators for oauth 2.0,” RFC 8707, IETF, 2020, <https://datatracker.ietf.org/doc/html/rfc8707>.
- [4] M. Jones, P. Hunt, and A. Parecki, “OAuth 2.0 protected resource metadata,” RFC 9728, IETF, 2025, <https://datatracker.ietf.org/doc/html/rfc9728>.
- [5] M. Jones, A. Nadalin, B. Campbell, J. Bradley, and C. Mortimore, “OAuth 2.0 token exchange,” RFC 8693, IETF, 2020, <https://datatracker.ietf.org/doc/html/rfc8693>.
- [6] IETF WIMSE WG, “Workload identity practices (wimse),” Internet-Draft, 2025, <https://datatracker.ietf.org/doc/draft-ietf-wimse-workload-identity-practices/>.
- [7] SPIFFE Project, “Spiffe: Secure production identity framework for everyone,” 2024, <https://spiffe.io/>.
- [8] MITRE, “Cwe-441: Unintended proxy or intermediary (confused deputy),” 2024, <https://cwe.mitre.org/data/definitions/441.html>.
- [9] N. Hardy, “The confused deputy: (or why capabilities might have been invented),” *ACM SIGOPS Operating Systems Review*, vol. 22, no. 4, pp. 36–38, 1988.
- [10] J. Richer, M. Jones, J. Bradley, M. Machulak, and P. Hunt, “OAuth 2.0 dynamic client registration protocol,” RFC 7591, IETF, 2015, <https://datatracker.ietf.org/doc/html/rfc7591>.
- [11] OWASP, “Owasp top 10 for large language model applications,” 2025, <https://owasp.org/www-project-top-10-for-large-language-model-applications/>.
- [12] OWASP GenAI Security Project, “Owasp top 10 risks & mitigations for agentic ai,” 2025, <https://genai.owasp.org/>.
- [13] D. Hardt, “The oauth 2.0 authorization framework,” RFC 6749, IETF, 2012, <https://datatracker.ietf.org/doc/html/rfc6749>.
- [14] Y. Sheffer, D. Hardt, and M. Jones, “Json web token best current practices,” RFC 8725, IETF, 2020, <https://datatracker.ietf.org/doc/html/rfc8725>.
- [15] IETF, OAuth WG, “On-behalf-of user authentication for ai agents in oauth 2.0,” Internet-Draft, 2025, <https://www.ietf.org/archive/id/draft-oauth-ai-agents-on-behalf-of-user-00.html>.